

Normative Support Layer

Degradation Signals and Fail-Closed Transitions v0.1

AGL - Runtime Narrowing, Hold, Freeze, and Quarantine Under
Grounding Loss

Status	Draft v0.1
Layer	Normative support layer / degradation discipline / fail-closed transition control
Canonical home	ester-reality-bound
Author	Kotov Ivan
Location	Brussels
Year	2026

Kotov Ivan
Bruxelles, 2026

Contents

Abstract	2
1. Purpose	2
2. Scope	2
3. Non-goals	3
4. Core principle	3
5. Why degradation needs its own layer	4
6. Degradation signal families	4
7. Degradation severity states	6
8. Transition rule set	6
9. Mapping degradation to runtime reliance	7
10. Mapping degradation to gate outcomes	8
11. Human degradation rules	8
12. Sensor and perception degradation rules	9
13. Local entity degradation rules	9
14. External / proxy degradation rules	10
15. Perimeter and environmental degradation rules	10
16. Recovery and re-grounding	11
17. Witness and persistence requirements	11
18. Relationship to ARL	11
19. Fail-closed rules	12
20. Failure modes addressed	12
21. Explicit bridge	13
22. Hidden bridges	13
23. Earth paragraph	13

Abstract

A long-lived system does not remain trustworthy merely because it once passed a grounding check.

Grounding can decay. Human attention can degrade. Sensors can drift. A proxy path can become detached from present execution. An authority can remain formally valid while operationally stale. A perimeter can stay “online” while already unsafe for reliance.

That is why the Actor Grounding Layer (AGL) cannot stop at qualification alone. It also needs explicit degradation logic.

This document defines:

- what counts as a degradation signal,
- how degradation should narrow runtime reliance,
- when degradation requires hold, freeze, or quarantine,
- why commit-time revalidation matters,
- and how a system remains fail-closed when grounding is lost after initiation but before consequence binds.

Its purpose is not to dramatize instability. Its purpose is to stop a serious system from converting degraded source-state into fluent continuation.

1. Purpose

To define:

- the classes of degradation signals relevant to AGL,
- the runtime consequences of each class,
- the transition logic from grounded states into narrowed or blocked states,
- the relation between degradation and runtime reliance,
- the difference between mild caution and hard stop,
- the recovery conditions required before degraded paths may widen again,
- and how degradation discipline interacts with ARL, L4 Witness, and hardware perimeter controls.

2. Scope

This document applies whenever a source that was visible, qualified, or provisionally accepted may lose grounding during:

- intake,
- pre-admissibility hold,
- action routing,
- privileged execution,
- review support,
- oracle or remote witness request,
- delayed re-entry,
- re-entry release,

- and commit-time consequence binding.

It applies across the four source classes already fixed in AGL:

- human / operator state,
- local entity initiation state,
- sensor / perception state,
- delegated / proxy / external origin state.

It also applies to runtime conditions that materially affect those sources:

- perimeter and hardware anomalies,
 - time-window drift,
 - budget exhaustion,
 - continuity fracture,
 - authority drift,
 - and environmental instability under L4.
-

3. Non-goals

This document does **not**:

- replace the AGL core,
- restate runtime reliance classes in full,
- restate gate families in full,
- replace ARL,
- replace L4 Witness,
- replace hardware perimeter controls,
- or reduce degradation to a single numeric score.

It also does not define UI mood indicators. If a system only “shows degradation” but does not narrow or stop runtime force, this layer has failed.

4. Core principle

4.1 Degradation is not commentary

A degradation signal is operational only if it changes what the system is allowed to rely on, authorize, or commit.

4.2 Degradation may narrow before it blocks

Not every degradation state requires immediate quarantine. But every material degradation must move runtime toward **equal or lower** reliance, never toward wider force.

4.3 Fail-closed transitions are monotonic under unresolved degradation

While degradation remains unresolved, allowed transitions may:

- narrow privilege,
- shorten windows,
- reroute into review,

- freeze,
- deny,
- or quarantine.

They must not silently widen by optimism.

4.4 Revalidation at commit is not optional for serious paths

A source may be grounded enough to initiate but no longer grounded enough when consequence actually binds.

4.5 Recovery is not the same thing as timeout expiry

A degraded path does not become healthy merely because time passed. It becomes healthier only when grounding is re-established through lawful evidence, present-state checks, or bounded requalification.

5. Why degradation needs its own layer

A recurring failure pattern in modern systems is this:

1. a source is qualified once,
2. the system begins to rely on it,
3. the source degrades,
4. later layers continue as if the original qualification still holds,
5. and the final action binds consequence under stale legitimacy.

This happens in ordinary life more often than people like to admit:

- a tired human initiates something serious,
- a stale sensor still “looks available,”
- a provider-mediated proxy still “looks connected,”
- a continuity-fractured local branch still “looks like the same system,”
- an escalation path remains open after the grounding basis that justified it has already decayed.

That is not a rare edge case. It is exactly where long-lived systems begin lying to themselves.

6. Degradation signal families

Degradation should be classed, not improvised.

6.1 Human / operator degradation signals

Examples include:

- overload,
- fatigue,
- sleep deprivation,
- panic,
- impaired attention,
- interpretive instability,

- compromised judgment under pressure,
- unstable role context,
- and state combinations where the human remains present but is no longer fit for ordinary high-impact reliance.

6.2 Local entity degradation signals

Examples include:

- drift accumulation,
- unresolved continuity fracture,
- deadlock pressure,
- witness-chain inconsistency,
- overloaded local reasoning loops,
- degraded internal state integrity,
- or local persistence that remains intact while present-state grounding does not.

6.3 Sensor / perception degradation signals

Examples include:

- stale sensor frames,
- degraded channel trust,
- missing telemetry,
- partial capture,
- environmental noise that destroys interpretive reliability,
- detached or replayed perception,
- or a path that is visible but too weak for runtime force.

6.4 Delegated / proxy / external origin degradation signals

Examples include:

- proxy-origin drift,
- delegated scope mismatch,
- provider-mediated abstraction,
- remote-origin lag,
- unverifiable present-state contact,
- replay or simulation surfaces,
- and authority that becomes thinner at every hop.

6.5 Perimeter / hardware degradation signals

Examples include:

- active anomaly flags,
- radio state or network posture inconsistent with required mode,
- maintenance mode leakage,
- unknown power-path condition,
- circuit-breaker activation,
- thermal instability,
- unexplained restarts,
- and physical access uncertainty.

6.6 Temporal / budget degradation signals

Examples include:

- expired review windows,
 - stale escalation basis,
 - action budget depletion,
 - token / spend budget breach,
 - retry budget exhaustion,
 - cooling windows not satisfied,
 - and commit that occurs after the lawful grounding moment has already passed.
-

7. Degradation severity states

This document recommends bounded severity states rather than fake precision.

7.1 NO_MATERIAL_DEGRADATION

The source remains sufficiently grounded for the requested current reliance class.

7.2 CAUTIONARY_DEGRADATION

The source remains usable, but runtime force must narrow. Typical effects:

- shorter window,
- stronger witness requirement,
- reduced privilege,
- or move from ordinary action to bounded action only.

7.3 HOLD_TRIGGERING_DEGRADATION

The source is no longer fit for ordinary progression and must enter hold or stronger qualification before continuing.

7.4 FREEZE_TRIGGERING_DEGRADATION

The source degradation is strong enough that contested mutation, escalation, or privileged movement must stop.

7.5 QUARANTINE_TRIGGERING_DEGRADATION

The degradation indicates contamination, continuity danger, unlawful re-entry risk, or source detachment strong enough that ordinary flow must isolate the path.

7.6 COMMIT_BLOCKING_DEGRADATION

The source may have been usable earlier, but is no longer grounded enough for consequence to bind now.

These are severity classes, not emotional tones.

8. Transition rule set

8.1 Degradation may only narrow or stop unresolved runtime force

Until a source is re-grounded, valid transitions must move toward lower runtime reliance.

8.2 No score soup

The system must not collapse degradation into one giant confidence score and pretend that arithmetic solved meaning.

8.3 Strongest relevant degradation wins for runtime force

If one source dimension remains healthy but another becomes severe enough to compromise lawful reliance, the stronger blocking consequence governs.

8.4 Commit-time degradation outranks intake optimism

If a source degrades before commit, the action must respect the **current** state, not the earlier cleaner state.

8.5 Recovery transitions require fresh grounding basis

A degraded path may widen only when fresh qualification or bounded revalidation lawfully supports it.

9. Mapping degradation to runtime reliance

This document assumes the runtime reliance classes defined elsewhere:

- RECORD_ONLY
- VISIBLE_BUT_NON_AUTHORITATIVE
- REVIEW_SUPPORT_ONLY
- ESCALATION_ELIGIBLE
- ACTION_ELIGIBLE_WITH_LIMITS
- COMMIT_ELIGIBLE
- RUNTIME_RELIANCE_DENIED

Recommended mapping:

9.1 NO_MATERIAL_DEGRADATION

May preserve the current lawful reliance class.

9.2 CAUTIONARY_DEGRADATION

Should usually move the source downward toward:

- ACTION_ELIGIBLE_WITH_LIMITS,
- ESCALATION_ELIGIBLE,
- or REVIEW_SUPPORT_ONLY.

9.3 HOLD_TRIGGERING_DEGRADATION

Should generally move the source toward:

- REVIEW_SUPPORT_ONLY,
- VISIBLE_BUT_NON_AUTHORITATIVE,
- or temporary RUNTIME_RELIANCE_DENIED until qualification is refreshed.

9.4 FREEZE_TRIGGERING_DEGRADATION

Should deny ordinary continuation and support:

- freeze entry,
- privilege narrowing,
- and at most REVIEW_SUPPORT_ONLY or RECORD_ONLY.

9.5 QUARANTINE_TRIGGERING_DEGRADATION

Should generally move the path to:

- RECORD_ONLY,
- VISIBLE_BUT_NON_AUTHORITATIVE,
- or RUNTIME_RELIANCE_DENIED,

with isolation of the affected branch, artifact, or execution surface.

9.6 COMMIT_BLOCKING_DEGRADATION

Must deny COMMIT_ELIGIBLE even if earlier stages passed.

10. Mapping degradation to gate outcomes

This document assumes the gate vocabulary defined elsewhere:

- gate-open,
- gate-open-with-limits,
- gate-reroute,
- gate-deny,
- gate-quarantine.

Recommended mapping:

10.1 Mild degradation

- gate-open-with-limits

10.2 Material unresolved degradation

- gate-reroute
- often into hold, review, or stronger qualification

10.3 Strong privilege or continuity degradation

- gate-deny
- Or gate-quarantine

10.4 Commit-time degradation

- gate-deny
- never “finish now and clean it up later”

11. Human degradation rules

Human-state degradation must remain explicitly in scope.

A human source may still be:

- the owner,
- the anchor,

- the responsible person,
- and yet be too degraded **now** for high-impact initiation or commit.

This is not a moral judgment. It is the same kind of operational sanity that prevents a serious system from treating:

- sleep deprivation,
- overload,
- panic,
- or broken attention

as if they were neutral execution conditions.

Where human degradation is material, the system should prefer:

- narrower privilege,
- stronger witness,
- shorter windows,
- hold,
- review support only,
- or denial of high-impact commit.

The system protects human agency by refusing to let bad state masquerade as ordinary authority.

12. Sensor and perception degradation rules

A sensor path that is available is not automatically grounded enough for action.

Where perception degrades materially, the system should:

- downgrade runtime force,
- widen uncertainty markers,
- preserve the path as trace if useful,
- and deny or reroute any action that depends on perceptual trust stronger than the current channel can honestly support.

A degraded sensor may still inform review. It must not silently authorize the world.

13. Local entity degradation rules

A long-lived local entity may remain continuity-bearing while still becoming too degraded for ordinary reliance.

This includes cases such as:

- unresolved drift,
- overloaded local loops,
- deadlock pressure,
- fractured continuity,
- contradictory prior state,

- or degraded internal integrity.

The system must not confuse persistence with present grounding.

Where the local entity degrades materially, actions should narrow toward:

- review support,
- hold,
- freeze,
- or quarantine,

not toward louder self-confidence.

14. External / proxy degradation rules

A common modern mistake is treating remote or proxy-origin signals as “cleaner” because they arrive through polished infrastructure.

That is often false.

Proxy and external-origin degradation may require stronger restrictions than local degradation because the present execution boundary becomes harder to verify.

Where proxy-origin grounding weakens, the default should move toward:

- escalation denial,
- review-only support,
- or runtime reliance denial.

Availability through a polished remote surface is not enough.

15. Perimeter and environmental degradation rules

The hardware perimeter is not background scenery. It is part of grounding.

If perimeter or environmental signals indicate:

- anomalous hardware posture,
- unstable power path,
- wrong network/radio mode,
- maintenance leakage,
- thermal distress,
- circuit-breaker condition,
- or other L4 contradictions,

then a source may become too degraded for ordinary reliance even if the higher-layer logic still looks coherent.

That is why perimeter degradation should be allowed to force:

- hold,
- freeze,
- commit denial,

- or quarantine,

without apologizing to interface smoothness.

16. Recovery and re-grounding

16.1 No auto-healing by silence

A degraded source does not recover merely because the system stopped talking about it.

16.2 Re-grounding must be explicit enough to matter

Recovery should rely on:

- fresh state qualification,
- present perimeter confirmation,
- refreshed privilege validity,
- bounded witness support,
- re-opened lawful windows,
- or other current operational evidence.

16.3 Recovery may still remain partial

A path may move from quarantine to visible-but-non-authoritative, or from denial to review-support-only, without returning to full action force.

16.4 No cosmetic restoration

The system must not simulate health by renaming degraded state as “good enough” while the underlying grounding remains unresolved.

17. Witness and persistence requirements

Material degradation transitions should not disappear into runtime fog.

Where degradation changes runtime force materially, the system should preserve enough durable record to show:

- what degraded,
- when,
- how runtime force narrowed,
- what path was denied or quarantined,
- and what fresh basis later widened it again, if that occurs.

This does not mean every micro-fluctuation needs public ceremony. It means that meaningful narrowing, denial, freeze, quarantine, and commit-block transitions should remain reconstructable.

18. Relationship to ARL

AGL degradation discipline sits **before** and **around** ARL.

ARL may later review conflict procedurally. But degradation logic explains why

some paths should:

- never reach ordinary continuation,
- enter hold early,
- arrive in review already narrowed,
- or remain quarantined even before a richer dispute frame is assembled.

In other words:

ARL handles conflict once procedural form exists. This document helps explain how a source may lose lawful runtime force **before** or **during** that process.

19. Fail-closed rules

19.1 No degradation, no widening by default

If degradation is unresolved, the system must not widen runtime force.

19.2 No stale authority under live degradation

Earlier legitimacy does not outrank present degradation at commit.

19.3 No commit under materially degraded grounding

If a source is too degraded to support consequence binding, the system must deny commit.

19.4 No elegant laundering of bad state

Smooth phrasing must not convert degraded grounding into acceptable runtime dependence.

19.5 No social override by presentation alone

Social or external input may matter greatly, but may not erase severe local degradation merely because the path sounds coherent from outside.

19.6 No hidden recovery

A blocked or narrowed path must not silently become ordinary again without lawful re-grounding.

20. Failure modes addressed

This layer is designed to reduce:

- stale-legitimacy execution,
 - commit under degraded source state,
 - human overload treated as ordinary authority,
 - available sensor paths mistaken for trustworthy perception,
 - proxy-origin polish mistaken for grounding,
 - local persistence mistaken for present integrity,
 - perimeter anomalies ignored by higher-level coherence,
 - and quiet restoration of blocked paths without fresh basis.
-

21. Explicit bridge

degradation signal <-> runtime narrowing <-> hold / freeze / quarantine <-> lawful re-grounding

This is the bridge that prevents grounding from becoming a one-time ceremonial check.

22. Hidden bridges

22.1 Hidden bridge #1 - ERB <-> SER

This layer explains why ARL intake, pre-admissibility hold, freeze, and re-entry discipline must remain sensitive to live degradation rather than only to static admissibility.

22.2 Hidden bridge #2 - ERB <-> ECC

This layer gives semantic force to runtime hooks, state persistence, witness events, and review routing in the executable bridge: without degradation transitions, those mechanics remain blind to the most common way legitimacy decays over time.

23. Earth paragraph

In a real workshop, a machine that passed inspection in the morning is not assumed safe for a critical cut at midnight just because the sticker is still on it. Bearings heat up, operators get tired, sensors drift, lubricants thin out, and the wrong part can still end up on the spindle. Serious shops do not call that “edge-case pessimism.” They call it shift reality. This layer brings the same adulthood to long-lived AI: qualification once is not enough if the source, the perimeter, or the actor has degraded before the cut actually happens.